

Flink state的优化 与remote state的探索

张杨 | bilibili资深开发工程师

01

相关背景

02

state压缩优化

03

remote state探索

04

未来规划

01 相关背景

业务概况

任务规模

4000+任务

95%sql类型

部署模式

80% on yarn

20% yarn on docker与在线混部

State使用情况

默认使用rocksdb statebackend

50%任务带状态

上百任务状态超过500GB

statebackend痛点

CPU抖动
大

大状态下compaction
引起cpu抖动大,
峰值资源要求高

恢复时间
长

大任务重启
或者任务rescale
状态恢复时间长

依赖本地
磁盘

强依赖本地io能力,
无法和在线业务机器
进行混部

优化思考



State压缩

- 降低compaction开销
- 关闭rocksdb底层压缩
- 支持业务层压缩



Remote state

- state存算分离
- state服务化
- state远程化

02 State压缩优化

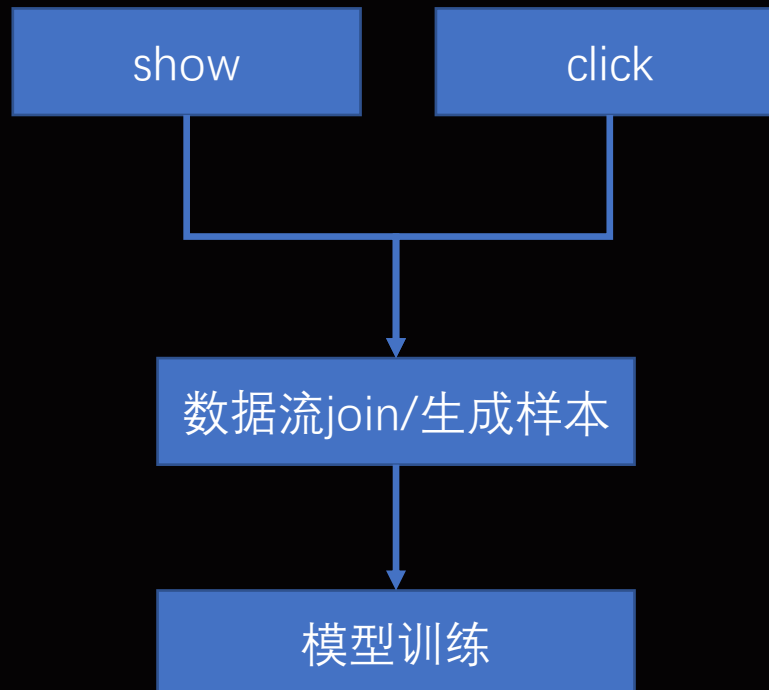
业务场景&问题

业务场景

- 模型训练，样本拼接
- key数量庞大，分布稀疏
- 数据ttl短，小时级



缓存失效快反复reload/compaction频繁/cpu消耗高

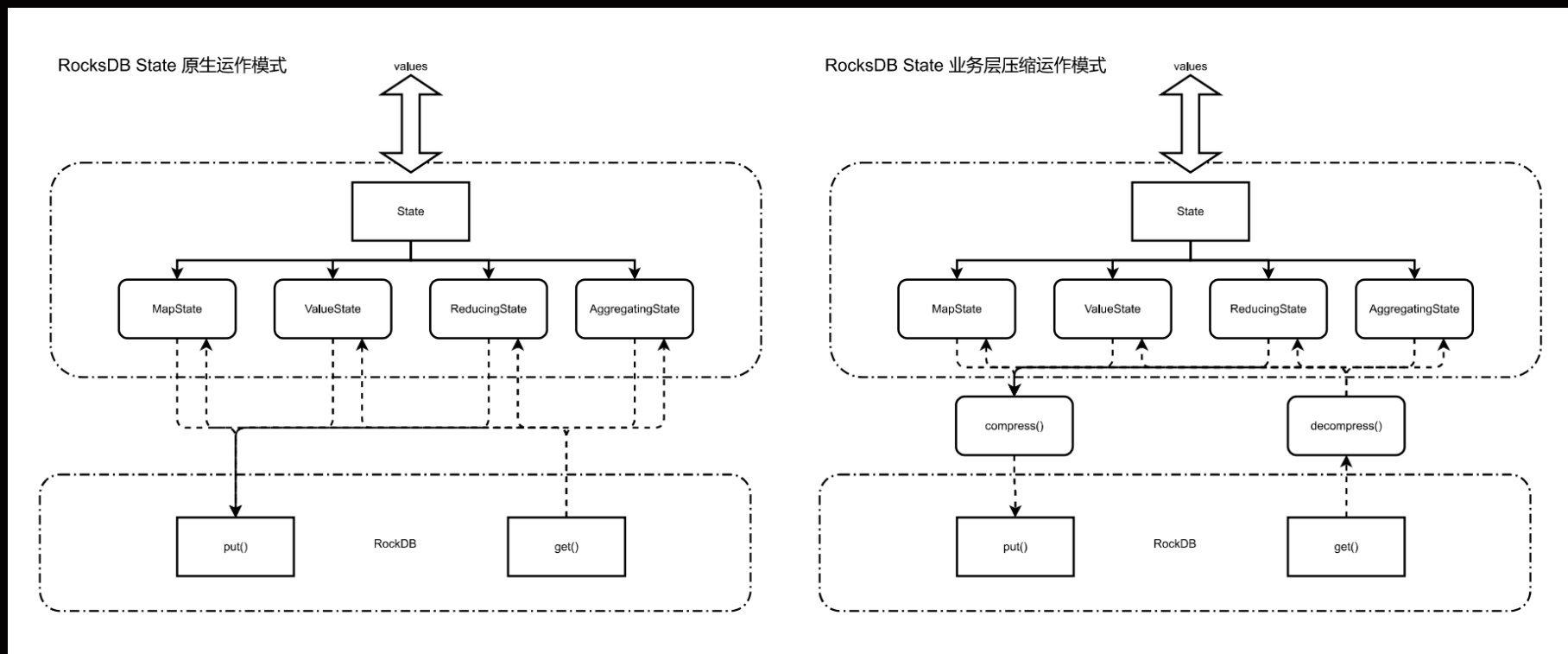


优化思路

- 开启partitioned index filter , 减少缓存竞争
- 关闭rocksdb压缩 , 减少缓存加载/compaction时候的cpu消耗
- 支持接口层压缩 , 在数据从state读写前后进行压缩解压缩操作 , 减少state大小

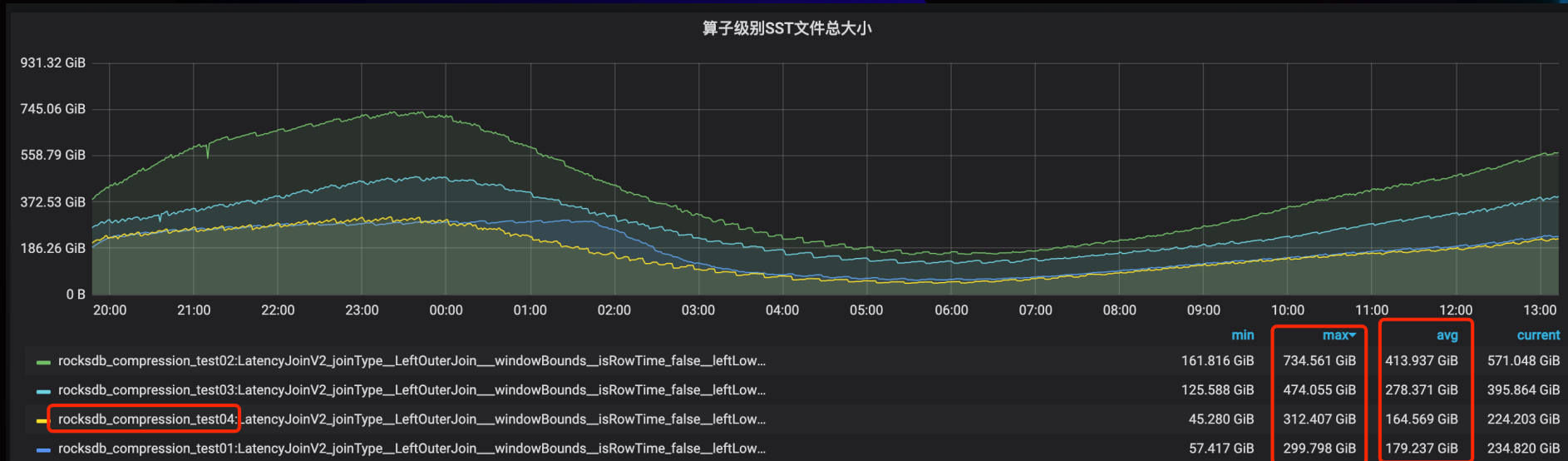
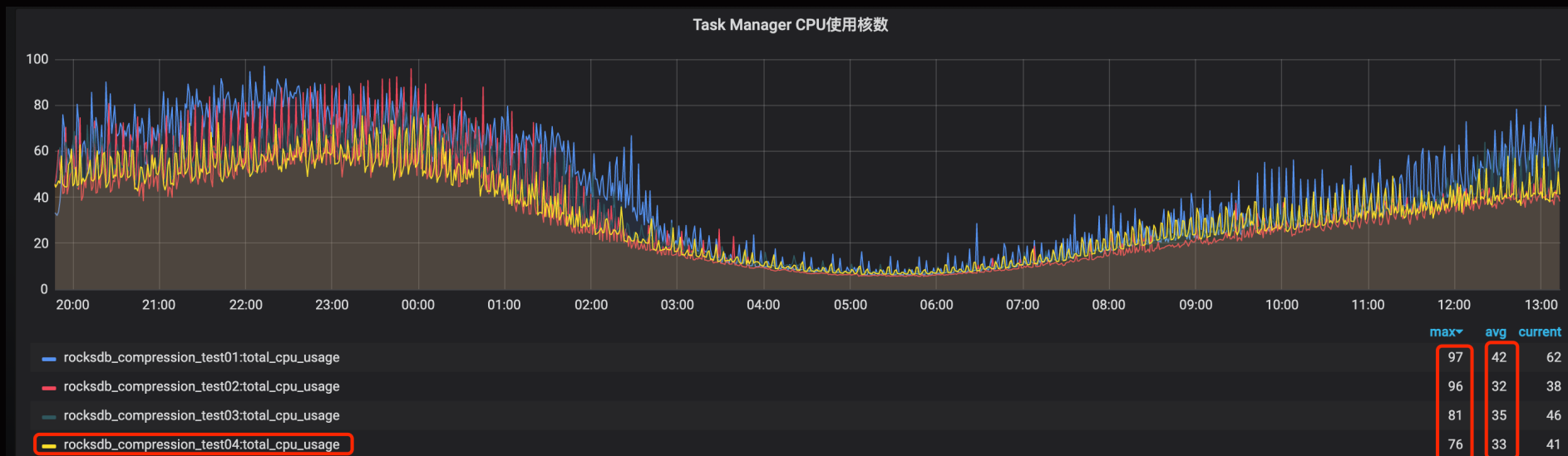
整体架构

- **partitioned index filter**开启以及rocksdb压缩关闭官方文档均有参数（版本高于1.12）
- 接口压缩架构如图,重写了所有rocksdb state的put/get接口,使用的是最新的Zstd压缩算法



业务效果

- cpu均值下降15%
- cpu峰值下降25%
- state文件大小基本不变



备注:

- 1)test01是社区默认
- 2)test04是业务层压缩

场景适应性说明

- 整体的压缩方案是完全匹配到我们的场景来的，主要双流join的大state场景，小state场景下基本没有收益（我们分析下来主要原因应该是小state的模式下，cache本身的命中率非常高，换进换出频率较低，compaction频率也比较低）。
- ListState暂时不支持业务层压缩，add接口底层的特殊merge实现暂时无法做业务层压缩

03 Remote state探索

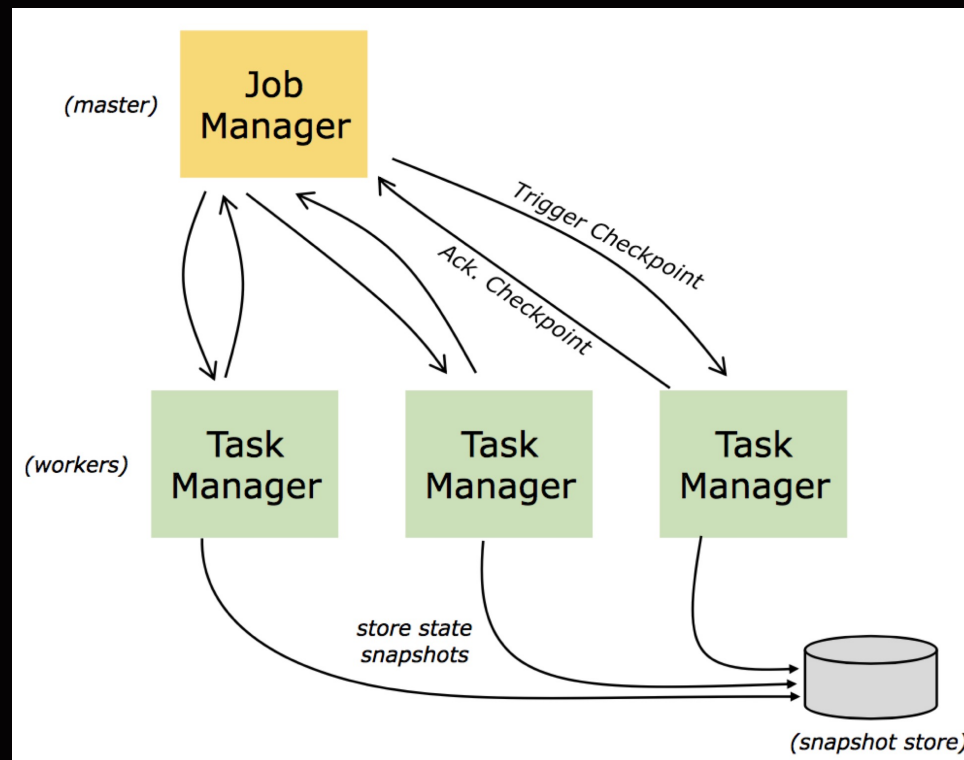
业务场景&问题

业务场景

- 离在线混部，降本增效
- 大state作业重启下载state慢
- 大state作业rescale加载state慢



在线业务机器io能力不足/state强依赖load过程

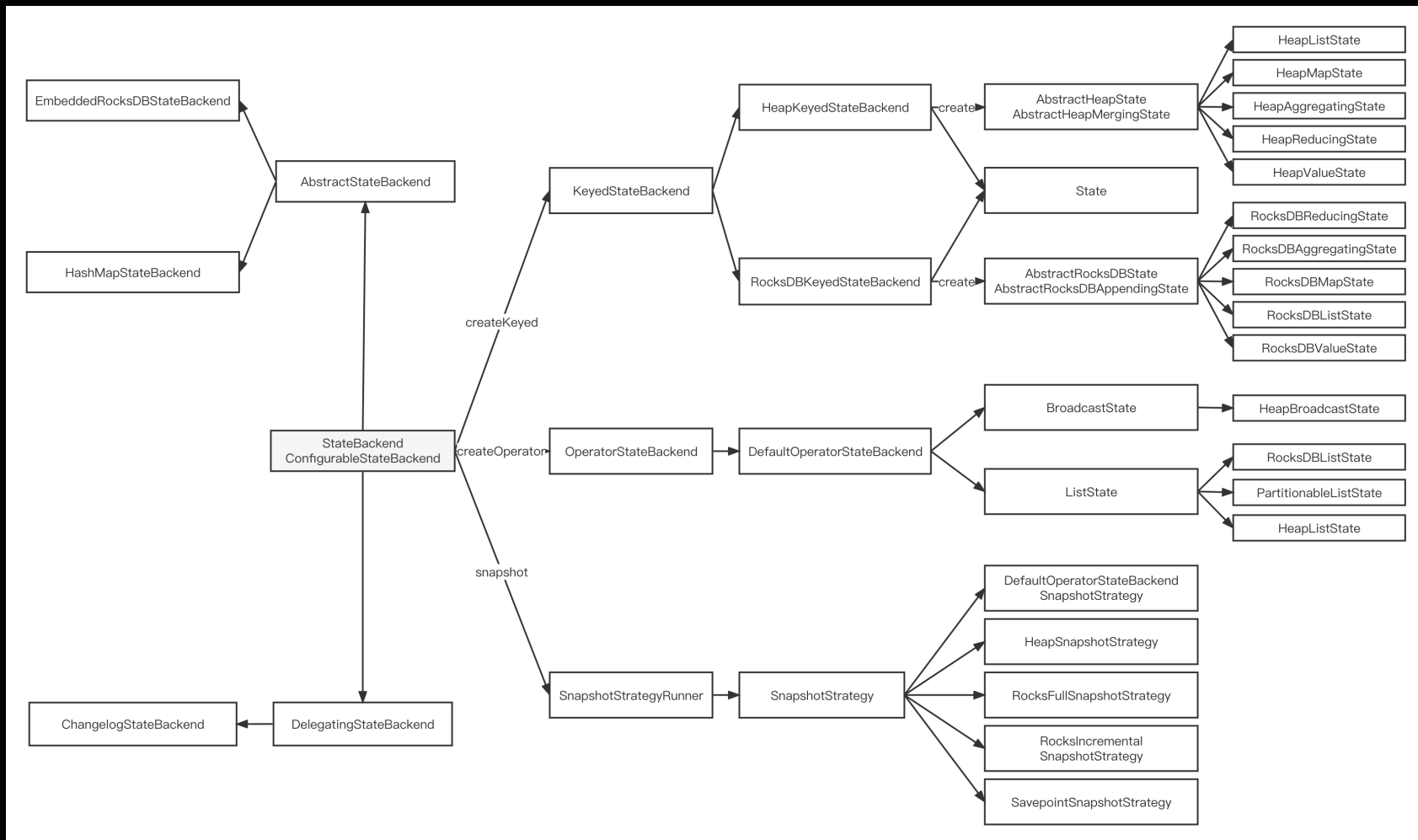


优化思路

- 实现flink state的存算分离
- State远程化/服务化，重启/rescale直接建立连接进行读写
- 远程state服务需要支持checkpoint完整功能依赖的接口

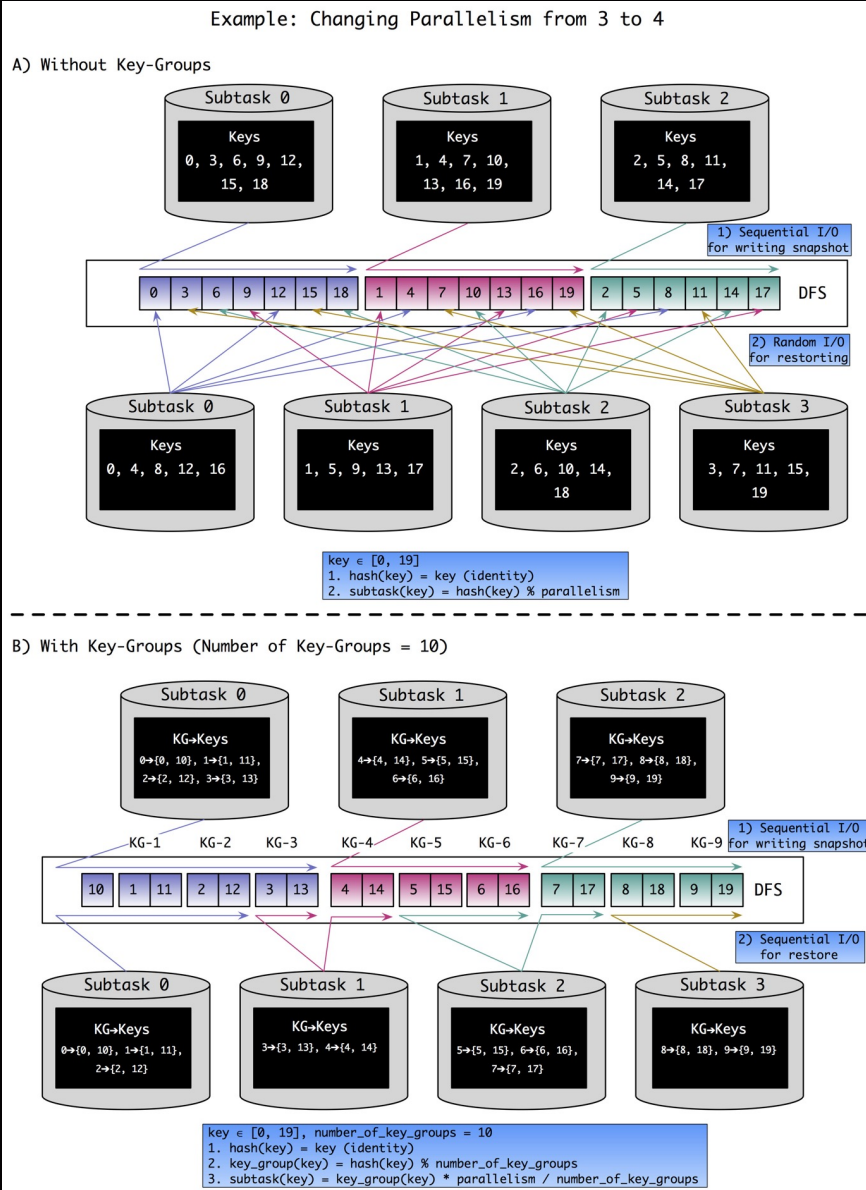
整体架构

- statebackend对象拓扑
- keyed statebackend承担核心的数据存储，有heap/rocksdb多种实现，支持大规模的数据存储



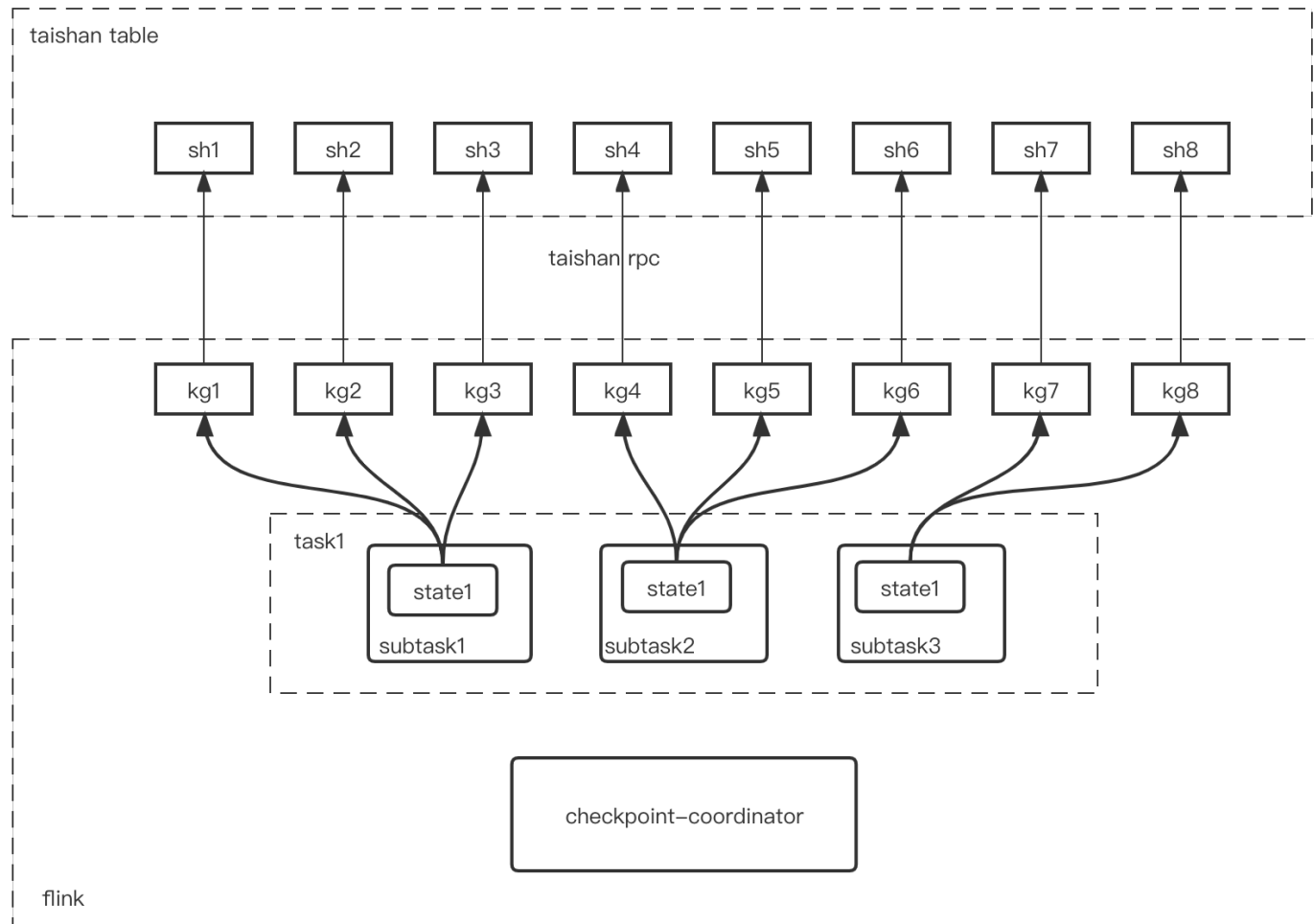
整体架构

- 基本的keyed statebackend分片逻辑
- 有限的rescale能力，本质是分片key-group在subtask之间的移动，分片无法分裂
- 多个key-group存储在相同rocksdb的cf里面，以key-group的id作为key的prefix，依赖rocksdb的排序特性，rescale过程通过prefix进行定位遍历提取



整体架构

- 替换rocksdb为b站内部的taishan存储
- key-group概念对应taishan存储的shard, cf对应table
- taishan存储支持对应shard的snapshot, 以及从历史snapshot进行fast-switch



缓存难点

写缓存

- 内存攒批，后台定期/checkpoint刷出去
- 上百倍的写rpc减少



效果好，达到预期效果

读缓存

- key较少场景效果明显，命中率极高，效果好
- 稀疏key场景，大量读null，缓存命中率低
- 周期性业导致缓存定期失效，读null，命中率暴跌



支持key全量缓存配置，有效解决稀疏/缓存失效场景性能问题，内存使用上升。

缓存难点

冷启动

- 缓存没有持久化
- 冷启动读请求大，存在性能压力
- 全量key缓存冷启动无法立即使用，需要等一个ttl周期



缓存持久化调研中

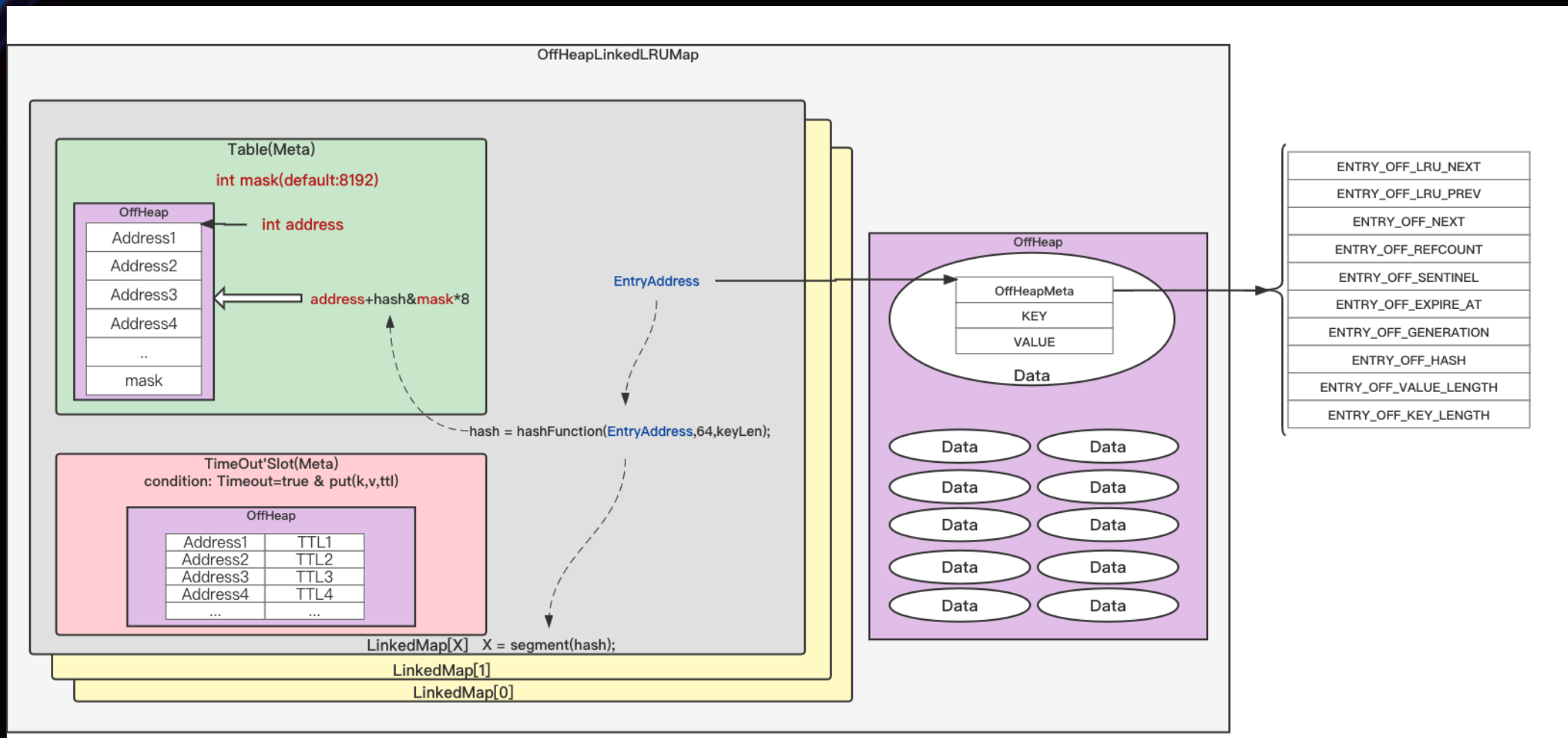
GC问题

- 缓存超过45%的jvm heap，频繁触发gc扫描
- Cache大量进入老年代，频繁触发mix-gc
- gc导致cpu消耗过高



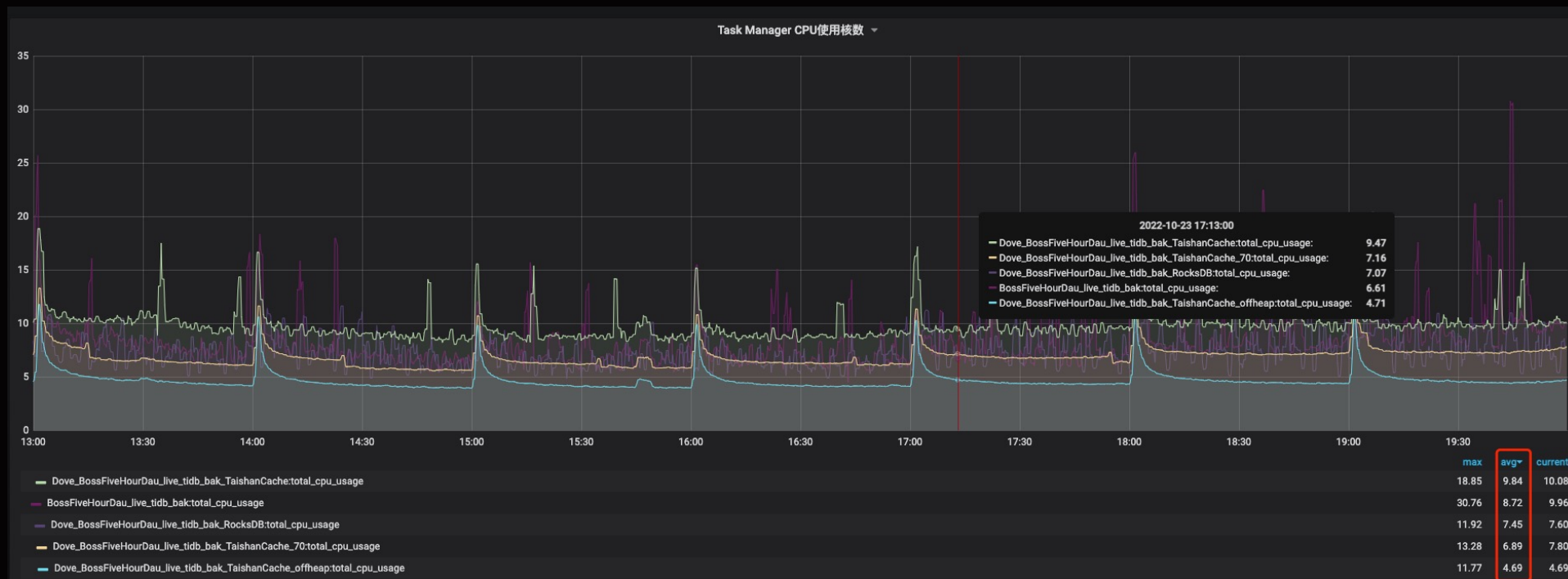
缓存从heap切换为off-heap，解决gc问题

off-heap缓存模型图



业务效果

- 目前我们在部分任务进行了灰度，灰度任务的计算资源消耗降低了30%左右
- 远程state的服务消耗暂时没有统计，粗略看整体的资源应该是持平的或者只有略微一点升高
- 离在线混部以及快速rescale的意义是巨大的



04 未来规划

state云原生推进

- 进一步优化remote statebackend，推进更多业务上线
- 推进flink云原生，提升离在线混部比例，支持云原生弹性伸缩

THANK YOU